

Universidad Tecnológica Nacional
Facultad Regional Bahía Blanca

Proyecto Final Capstone

Consumo y visualización de datos desde una API en contenedor

Informe Técnico

Asignatura: Arquitectura y Sistemas Operativos
Tecnicatura Universitaria en Programación

Integrantes:

Luca Farias
Bruno Verna

Fecha de entrega: 1 de agosto de 2025

1. Antecedentes

El presente proyecto se desarrolla como trabajo final integrador de la asignatura Arquitectura y Sistemas Operativos, correspondiente a la Tecnicatura Universitaria en Programación de la UTN FRBB. Su objetivo es integrar los contenidos abordados durante la cursada —administración de procesos, gestión de memoria, redes, sistemas distribuidos y, en particular, virtualización y contenedores— en una solución práctica orientada al tratamiento de datos generados por un dispositivo IoT real.

La propuesta combina el uso de Docker para la contenerización de servicios con Node-RED como plataforma de procesamiento de flujos de datos, permitiendo consumir información desde una API REST pública, transformarla y visualizarla en tiempo real a través de un dashboard web.

2. Descripción de la actividad

El sistema desarrollado consume periódicamente el endpoint público correspondiente a un dispositivo IoT operativo:

```
https://callback-iot.up.railway.app/data
```

Sobre esta información se configuró un flujo en Node-RED que realiza una consulta automática cada 10 segundos, ordena los registros recibidos por fecha y hora, extrae los dos más recientes y los representa visualmente mediante widgets de tipo Gauge y Chart dentro del Dashboard de Node-RED. Toda la solución se empaquetó dentro de un contenedor Docker, gestionado mediante un archivo docker-compose.yml que permite levantar el sistema completo con un solo comando.

3. Marco teórico

3.1 ¿Qué es Docker y para qué se utiliza?

Docker es una plataforma de código abierto que permite crear, ejecutar y distribuir aplicaciones dentro de contenedores de forma aislada y reproducible. Surgida en 2013, automatiza el despliegue de aplicaciones proporcionando una capa adicional de abstracción y automatización de virtualización a nivel de sistema operativo.

A diferencia de una máquina virtual —que requiere un sistema operativo completo (Guest OS) corriendo sobre un hipervisor—, un contenedor Docker comparte el kernel del sistema anfitrión y aísla únicamente los procesos y el sistema de archivos de la aplicación. Esto lo hace más liviano, rápido de iniciar y eficiente en recursos. Como se vio en clase: si una máquina virtual es toda una casa, un contenedor es una habitación con lo justo para vivir.

Los elementos clave del ecosistema Docker son:

- Imagen: plantilla inmutable para crear contenedores, construida a partir de un Dockerfile.

- Contenedor: instancia activa y aislada de una imagen; es lo que realmente se ejecuta.
- Dockerfile: instrucciones para construir una imagen de forma automatizada y reproducible.
- docker-compose.yml: archivo que define y ejecuta múltiples contenedores como un sistema integrado.

La filosofía de Docker establece que en cada contenedor debe correr un único servicio o proceso, reduciendo la complejidad respecto a las máquinas virtuales. En este proyecto, el docker-compose.yml levanta Node-RED como servicio contenerizado, exponiendo el puerto 1880 y persistiendo la configuración mediante un volumen, garantizando que el flujo funcione igual en cualquier máquina.

3.2 ¿Qué es una API REST y cómo se consulta?

Una API (Application Programming Interface) REST es una interfaz que permite la comunicación entre sistemas a través del protocolo HTTP, siguiendo los principios de la arquitectura REST (Representational State Transfer): comunicación sin estado entre cliente y servidor, uso de métodos HTTP estándar (GET, POST, PUT, DELETE) y representación de los recursos típicamente en formato JSON.

En este proyecto, el dispositivo IoT expone sus datos a través de un endpoint HTTP que responde a solicitudes GET devolviendo un arreglo JSON con los registros generados por el sensor. La consulta se realiza mediante el nodo http request de Node-RED, que efectúa la petición GET, espera la respuesta del servidor y entrega el cuerpo de la respuesta (ya parseado como objeto JavaScript) al resto del flujo para su procesamiento.

3.3 Características de un sistema IoT y tipo de datos que genera

Internet of Things (IoT) hace referencia a redes de dispositivos físicos —sensores, actuadores, microcontroladores— equipados con la capacidad de recolectar, transmitir y, en algunos casos, procesar datos a través de internet sin intervención humana directa. Entre sus características generales se destacan:

- Conectividad permanente o periódica hacia internet o una red local.
- Generación continua de datos en tiempo real, habitualmente con marca de tiempo (timestamp).
- Bajo consumo de recursos computacionales en el dispositivo (hardware embebido, microcontroladores).
- Comunicación frecuente mediante protocolos livianos como HTTP, MQTT o CoAP.
- Los datos generados suelen ser mediciones numéricas de variables físicas (temperatura, humedad, presión, distancia, niveles, etc.), acompañadas de metadatos como el identificador del dispositivo y la marca temporal de la lectura.

El dispositivo consultado en este proyecto sigue este patrón: expone sus lecturas como una colección de registros en formato JSON, cada uno asociado a un instante de tiempo, lo cual permite ordenar cronológicamente las muestras y quedarnos con las más recientes para su visualización.

3.4 Rol de Node-RED en esta arquitectura

Node-RED es una herramienta de programación visual basada en flujos (flow-based programming), originalmente desarrollada por IBM, pensada para conectar dispositivos de hardware, APIs y servicios online entre sí. Permite construir aplicaciones de integración de datos mediante el armado de flujos compuestos por nodos —cada uno con una función específica: recibir datos, transformarlos, enviarlos a otro sistema o representarlos visualmente— conectados entre sí mediante cables (wires) en una interfaz web.

En la arquitectura de este proyecto, Node-RED actúa como capa de procesamiento intermedio: se encarga de disparar la consulta periódica al endpoint (nodo inject), realizar la petición HTTP (nodo http request), aplicar lógica de transformación de datos mediante JavaScript (nodo function, donde se ordenan los registros por fecha y se extraen los dos más recientes) y finalmente entregar la información procesada a los widgets del Dashboard (Gauge y Chart) para su visualización en tiempo real. De esta manera, cumple un rol equivalente al de un middleware ligero entre la fuente de datos IoT y la capa de presentación.

4. Desarrollo de la actividad

4.1 Análisis del endpoint y formato del JSON

El primer paso consistió en analizar la respuesta del endpoint público mediante una consulta GET directa desde el navegador y desde Node-RED, para conocer la estructura del JSON devuelto. El dispositivo IoT responde con un arreglo de objetos, donde cada objeto representa una lectura del sensor en un instante determinado. Un ejemplo representativo de la estructura de datos esperada es el siguiente:

```
[
  {
    "device": "42A6DA",
    "timestamp": "2025-07-30T14:25:10.693Z",
    "temperature": 8.25,
    "humidity": 59.8,
    "pressure": 1029.28
  },
  {
    "device": "42A6DA",
    "timestamp": "2025-07-30T14:25:20.100Z",
    "temperature": 8.30,
    "humidity": 60.1,
    "pressure": 1029.31
  }
]
```

The screenshot displays the Node-RED web interface in a browser. The top section shows a flow configuration for 'IoT Capstone - Luca'. The flow starts with a 'Cada 10 segundos' (Inject) node, followed by a 'GET /data' (HTTP Request) node, then a function node 'Ordenar y extraer 2 mas recientes'. This function node has two outputs: one leading to 'Preparar valor para Gauge/Chart' (function) and another to 'JSON crudo (2 mas recientes)' (JSON Output). The 'Preparar valor para Gauge/Chart' node connects to a 'Gauge sensor' and a 'Chart historico' node. The 'JSON crudo' node connects to a 'JSON Output' node. The right sidebar shows the 'Flow' tab with the configuration nodes listed. Below the flow configuration, the 'DEBUG MESSAGES' panel shows a series of error messages: 'El endpoint no devolvió datos en este momento'.

Below the Node-RED interface, the 'Proyecto Capstone IoT' dashboard is visible. It features a 'Dashboard IoT' title and a 'Valor del sensor' gauge showing a value of 0. The gauge has a scale from 0 to 100. Below the gauge is a 'Historico del sensor' section with the text 'Esperando datos...'.

4.2 Diseño del flujo en Node-RED

El flujo implementado se compone de los siguientes nodos, en orden de ejecución:

1. Inject (cada 10 segundos): dispara la ejecución del flujo de forma periódica y automática, sin intervención manual.
2. HTTP Request (GET /data): realiza la consulta al endpoint del dispositivo IoT y entrega la respuesta JSON al siguiente nodo.

3. Function — “Ordenar y extraer 2 más recientes”: valida que la respuesta sea un arreglo no vacío, ordena los registros por fecha/hora de forma descendente (del más reciente al más antiguo) y se queda únicamente con los dos primeros.
4. Function — “Preparar valor para Gauge/Chart”: toma el registro más reciente y extrae el valor numérico del sensor para alimentar los widgets de visualización.
5. Debug: muestra en la consola lateral de Node-RED el JSON completo de los dos registros más recientes, útil para verificar que el procesamiento es correcto.
6. Gauge y Chart (Dashboard): widgets que muestran, respectivamente, el valor actual del sensor en un indicador tipo velocímetro y su evolución histórica en un gráfico de líneas.

El código del nodo function principal, encargado de ordenar los datos por fecha y quedarse con los dos registros más recientes, es el siguiente:

```
let datos = msg.payload;

if (!Array.isArray(datos) || datos.length === 0) {
  node.warn("El endpoint no devolvió datos en este momento");
  return null;
}

datos.sort((a, b) => {
  let fechaA = new Date(a.timestamp || a.fecha || a.createdAt || a.date);
  let fechaB = new Date(b.timestamp || b.fecha || b.createdAt || b.date);
  return fechaB - fechaA;
});

let ultimosDos = datos.slice(0, 2);

msg.payload = ultimosDos;
return msg;
```

El archivo flujo-nodered.json (incluido en el repositorio) contiene la definición completa del flujo y puede importarse directamente desde el menú Import del editor de Node-RED.

4.3 Capturas del flujo implementado

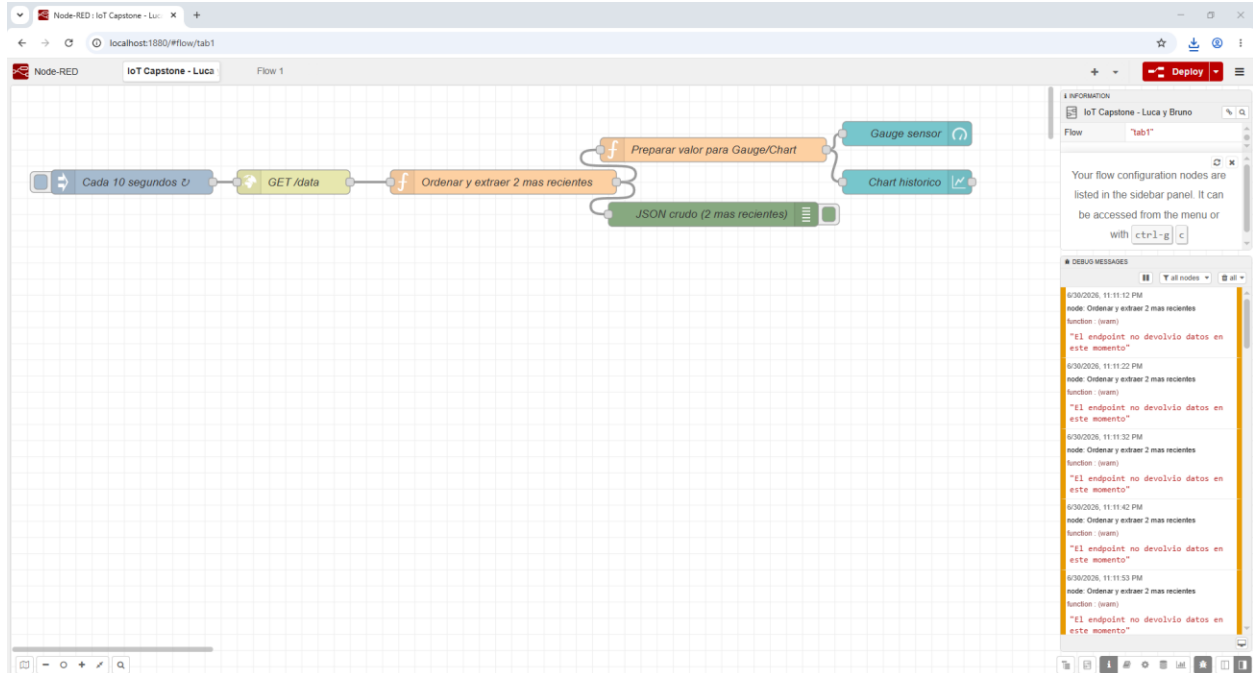


Figura 1. Flujo completo implementado en Node-RED.

4.4 Capturas de los datos visualizados

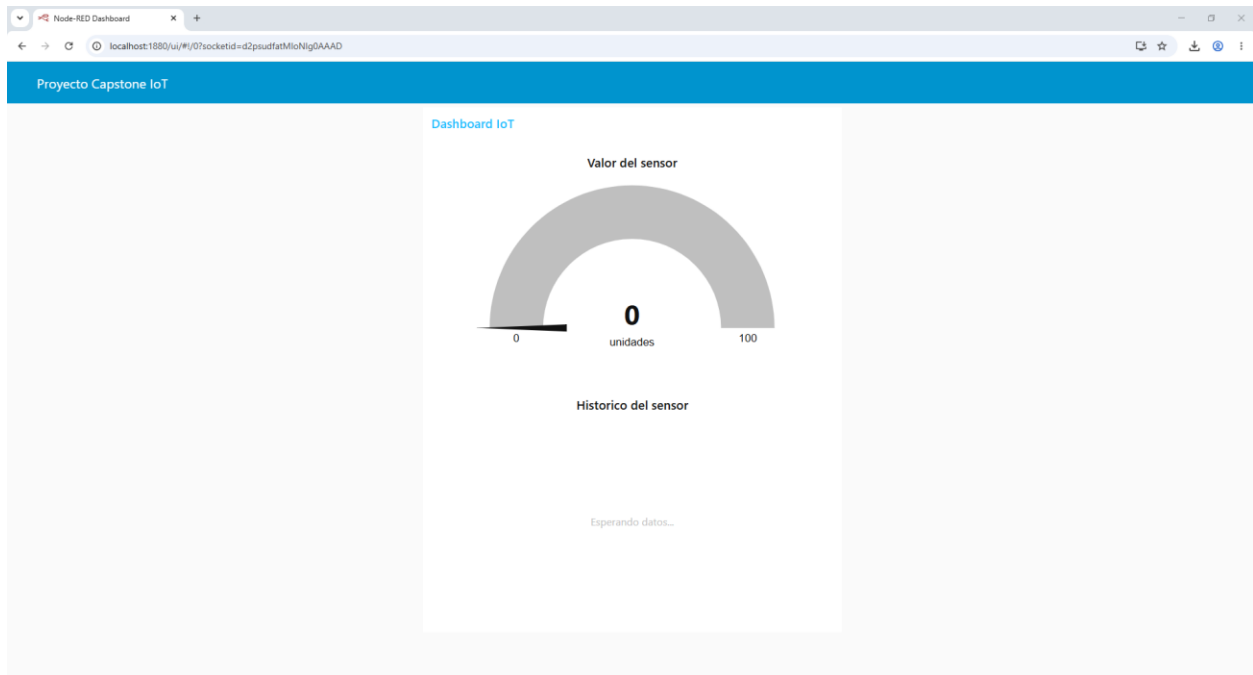


Figura 2. Dashboard de Node-RED .

4.5 Contenerización con Docker

La solución completa fue empaquetada en un contenedor Docker a partir de la imagen oficial `nodered/node-red`, exponiendo el puerto 1880 (editor y dashboard) y persistiendo la configuración

mediante un volumen Docker, de forma que los flujos no se pierdan al reiniciar el contenedor. El archivo docker-compose.yml utilizado es el siguiente:

```
version: "3.8"

services:
  node-red:
    image: nodered/node-red:latest
    container_name: nodered-iot
    restart: unless-stopped
    ports:
      - "1880:1880"
    volumes:
      - node-red-data:/data
    environment:
      - TZ=America/Argentina/Buenos_Aires

volumes:
  node-red-data:
```

Para levantar el sistema completo, basta con ejecutar desde la raíz del repositorio:

```
docker-compose up -d
```

Una vez levantado el contenedor, el editor de Node-RED queda disponible en <http://localhost:1880> y el dashboard de visualización en <http://localhost:1880/ui>.

5. Diagrama de bloques del sistema

El siguiente diagrama representa de forma esquemática la arquitectura completa del sistema desarrollado, desde el origen de los datos hasta su visualización final:

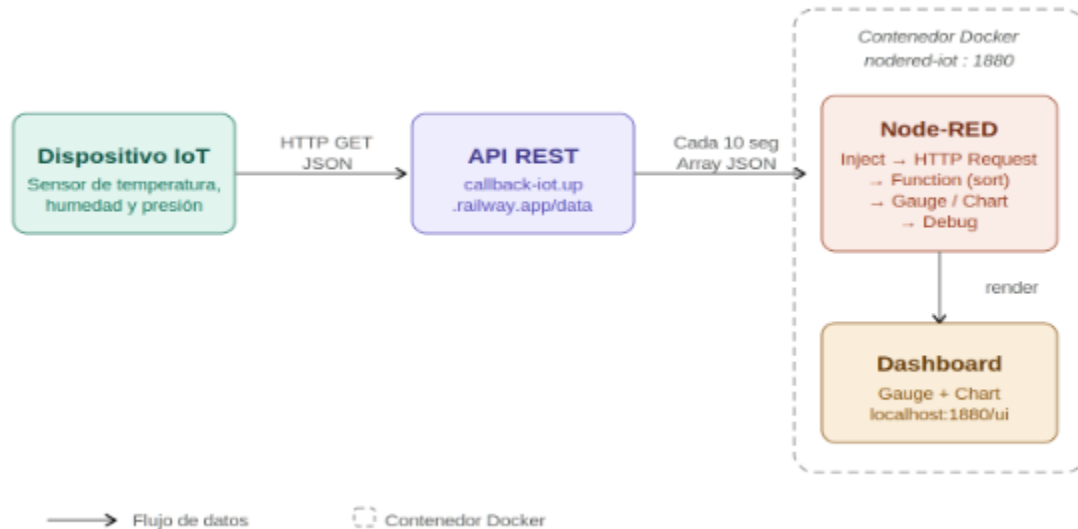


Figura 3. Diagrama de bloques del sistema IoT desarrollado.

El diagrama debe representar claramente cuatro etapas, conectadas mediante flechas que indiquen el sentido del flujo de datos:

- Origen de los datos: dispositivo IoT, que genera lecturas periódicas de un sensor.
- Transporte y formato: exposición de los datos mediante una API REST, en formato JSON, accesible vía HTTP GET.
- Procesamiento intermedio: flujo en Node-RED (ejecutado dentro de un contenedor Docker), encargado de consultar el endpoint, ordenar los registros y extraer los más recientes.
- Capa de visualización: Dashboard web de Node-RED, con widgets Gauge y Chart mostrando los datos procesados en tiempo real.

Sugerencia de herramienta para construir el diagrama: draw.io (app.diagrams.net), Excalidraw o Lucidchart, exportando el resultado como imagen PNG.

6. Reflexión sobre dificultades y aprendizajes

El desarrollo de este proyecto implicó enfrentarse a desafíos concretos que nos permitieron consolidar los contenidos teóricos abordados durante la cursada en un contexto práctico real.

Una de las primeras dificultades fue la instalación y configuración de Docker Desktop en nuestros equipos. Si bien la herramienta está bien documentada, comprender la diferencia entre imagen, contenedor y volumen —conceptos que en clase habíamos visto de forma teórica— tomó un tiempo de práctica hasta que el modelo mental quedó claro. En particular, la analogía que vimos en clase ayudó mucho: pensar en el contenedor como 'una habitación con lo justo para vivir', en contraste con la máquina virtual que sería 'toda una casa', hizo que la diferencia de liviandad y aislamiento se entendiera de forma inmediata.

El segundo desafío fue trabajar con el formato de respuesta del endpoint. Al realizar las primeras pruebas, el endpoint devolvía un arreglo vacío ([]) en algunos momentos, lo que inicialmente nos generó confusión sobre si el flujo estaba funcionando correctamente o si había un error de configuración. Investigando en la documentación de Node-RED y revisando los logs del contenedor con `docker logs nodered-iot`, comprendimos que el dispositivo IoT no transmite datos de forma continua, sino en intervalos de tiempo específicos. A partir de ese momento diseñamos el nodo function para manejar el caso del arreglo vacío de forma elegante, retornando null y emitiendo una advertencia en el panel de debug sin que el flujo se rompa.

El manejo de fechas en JavaScript dentro del nodo function fue otro punto que requirió atención: los timestamps del endpoint vienen en formato ISO 8601 (por ejemplo, '2025-07-30T14:25:10.693Z'), y el método `Date.parse()` los maneja correctamente. Sin embargo, fue necesario validar que el campo de fecha existiera antes de intentar convertirlo, para evitar errores silenciosos cuando algún registro venía incompleto.

En cuanto a lo aprendido, este proyecto nos permitió ver en funcionamiento real el ciclo completo de un sistema IoT: desde la generación de datos en un sensor físico, su exposición a través de una API REST en formato JSON, el consumo y procesamiento de esos datos con lógica JavaScript dentro de un flujo de Node-RED, hasta la visualización en tiempo real en un dashboard web. Todo esto corriendo de forma aislada y reproducible dentro de un contenedor Docker.

El proyecto también nos hizo ver la conexión directa con los contenidos teóricos de la materia: los contenedores son una forma concreta de aplicar los conceptos de aislamiento de procesos y gestión de recursos vistos en las unidades de sistemas operativos; la API REST es un ejemplo práctico de comunicación entre procesos en red; y Node-RED nos dio visibilidad sobre cómo funciona un sistema de orquestación de servicios distribuidos a pequeña escala.

Como mejoras futuras, identificamos las siguientes posibilidades: persistir el historial de lecturas en una base de datos (por ejemplo, InfluxDB, que se integra fácilmente con Node-RED y Docker); incorporar alertas automáticas cuando algún valor supera un umbral configurable; agregar autenticación al endpoint para proteger el acceso a los datos del dispositivo; y eventualmente desplegar la solución en la nube (por ejemplo, en Railway o Render) para que el dashboard sea accesible públicamente sin depender de que el equipo local esté encendido.

7. Conclusión

El desarrollo de este proyecto permitió integrar de forma práctica los conceptos teóricos abordados durante la cursada de Arquitectura y Sistemas Operativos, particularmente en lo referido a virtualización y contenedores, aplicándolos a un caso de uso real de consumo y visualización de datos IoT. La combinación de Docker como tecnología de empaquetado y despliegue, junto con Node-RED como plataforma de orquestación de flujos de datos, demuestra cómo herramientas modernas de virtualización facilitan la construcción de sistemas distribuidos reproducibles, portables y de rápida puesta en marcha.

8. Referencias

Docker Inc. Documentación oficial de Docker. Disponible en: <https://docs.docker.com/>

Node-RED Project. Documentación oficial de Node-RED. Disponible en: <https://nodered.org/docs/>

Node-RED Dashboard. Documentación de los nodos de visualización. Disponible en:

<https://flows.nodered.org/node/node-red-dashboar>

IBM. Node-RED: documentación y guía de inicio. Disponible en: <https://nodered.org/docs/getting-started/>